

Lecture 9

Unity 스크립트로서의 C#

일반 C# 프로그램과 Unity 스크립트의 차이

강영민

동명대학교 게임학부

목차

- 1 이번 강의의 출발점
- 2 Unity 스크립트의 기본 구조
- 3 MonoBehaviour의 생명 주기
- 4 컴포넌트 기반 설계
- 5 Inspector와 직렬화
- 6 주요 고려 사항과 함정
- 7 입력과 충돌 처리
- 8 실전 예제: 충돌하면 합쳐지는 공
- 9 단계별 도전 과제
- 10 정리

이번 강의에서 다루는 것

학습 목표

- 일반 C# 프로그램과 Unity 스크립트의 **진입 방식**이 어떻게 다른지 이해한다
- MonoBehaviour와 **생명 주기 메서드**의 호출 시점을 파악한다
- Unity의 **컴포넌트 기반 설계**를 이해한다
- Unity의 **Inspector 직렬화**와 노출 규칙을 안다
- Unity 스크립트 작성 시 흔한 **합정과 성능 규칙**을 피할 수 있다

대상

이미 일반 C# 프로그래밍의 기초(클래스, 상속, 메서드, 제어문, 컬렉션 등)는 알고 있는 학생. Unity는 처음 접하더라도, “같은 C# 언어인데 왜 작성 방식이 이렇게 달라 보이는가”에 답을 얻는 것이 목표.

일반 C# 프로그램에서 우리가 직접 했던 일들

⚙️ 콘솔/GUI 프로그램에서 우리가 짰 코드

- **진입점** Main 작성 — 프로그램이 어디서 시작되는가
- 객체를 new로 직접 생성
- 입력 처리 — `Console.ReadLine`, 또는 GUI라면 `KeyDown` 이벤트
- 출력 처리 — `Console.WriteLine`, 또는 GUI라면 `Graphics`로 직접 그리기
- 시간 흐름 처리 — 타이머 또는 `Thread.Sleep`으로 간격 만들기
- 게임이라면 **게임 루프**를 직접 작성 (위치 갱신 → 충돌 검사 → 다시 그리기)

핵심

Unity에서는 **이 모든 것이 이미 만들어져 있다.**
우리는 `Update()`처럼 **약속된 이름**의 메서드만 채워 넣는다.

프레임워크 vs 라이브러리

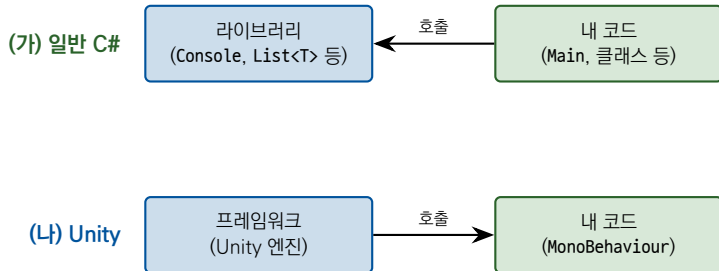
↔ 호출 방향의 “역전”

- **일반 C# 프로그램**: 우리가 라이브러리를 호출한다.
`Console.WriteLine(...), list.Add(...)`
- **Unity 스크립트**: 프레임워크가 우리를 호출한다.
`void Update() { ... }` ← Unity가 매 프레임 자동 호출

헐리우드 원칙 (Hollywood Principle)

이 “역전된 호출 관계”를 **IoC(Inversion of Control)** 또는 **헐리우드 원칙**(“Don’t call us, we’ll call you”)이라 부른다.
프레임워크가 “Update라는 이름의 메서드는 매 프레임 호출하겠다”는 약속을 정해 두면, 우리는 그 약속에 맞춰 메서드를 작성하
기만 하면 된다.

개념도: 라이브러리 vs 프레임워크



💡 한 줄 요약

라이브러리는 “내가 부른다”, 프레임워크는 “나를 부른다”.

가장 기본적인 Unity 스크립트

HelloUnity.cs

```
1 using UnityEngine;
2
3 public class HelloUnity : MonoBehaviour
4 {
5     void Start() { Debug.Log("Hello, Unity!"); }
6     void Update() { /* Called every frame */ }
7 }
```

핵심 4가지

- `using UnityEngine;` — Unity 기본 네임스페이스
- `: MonoBehaviour` — Unity가 인식하는 “스크립트 컴포넌트”가 되기 위한 필수 상속
- 클래스 이름 = **파일 이름**(HelloUnity.cs)이어야 함
- `Debug.Log` — Unity Console에 출력 (`Console.WriteLine`의 Unity 버전)

MonoBehaviour란 정확히 무엇인가?

Unity가 미리 만들어 둔 기반 클래스

- 미리 정의된 “빈” 메서드들 — 약 30개
Awake, Start, Update, OnCollisionEnter ...
기본 구현은 “아무 것도 안 함”. 우리가 채워 넣는다.
- 유용한 속성들
transform, gameObject, enabled, name ...

주의: override 키워드를 쓰지 않는다

Unity는 메서드 이름을 보고 리플렉션으로 찾아 호출한다. 그래서 override 키워드가 필요 없지만, 반대로 **메서드 이름의 철자가 틀리면 그냥 호출되지 않을 뿐** 컴파일 에러는 나지 않는다. (Updaet, start 같은 실수 주의)

Main이 없다는 게 무슨 뜻인가?

▶ Play 버튼을 누르면 Unity가 하는 일

- 1 현재 **Scene**에 있는 모든 **GameObject**를 찾는다
- 2 각 **GameObject**에 붙어 있는 **MonoBehaviour**를 찾는다
- 3 각 스크립트의 **Awake** → **OnEnable** → **Start**를 차례로 호출
- 4 매 프레임마다 모든 활성 스크립트의 **Update**를 호출

💡 한 줄 요약

Unity 엔진 자체가 거대한 **Main**이라고 생각하면 된다.

우리는 그 안에서 “특정 시점에 실행될 코드”만 약속된 이름의 메서드에 작성한다.

Scene(씬)이란?

≡ “한 화면을 구성하는 GameObject들의 모음”

- TitleScene — 타이틀 화면 (로고, 시작 버튼)
- StageScene — 게임 플레이 화면 (플레이어, 적, UI)
- EndingScene — 엔딩 화면

파일로 저장됨 (.unity 확장자).

`SceneManager.LoadScene("StageScene")`으로 전환.

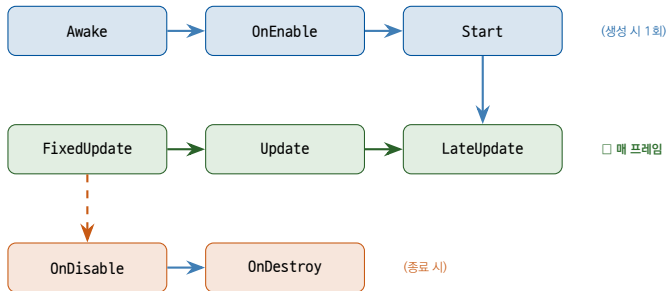
↔ 일반 GUI 프로그램과의 비교

일반 GUI 프로그램에서는 “하나의 폼/창”을 띄우고 그 안에서 화면을 직접 갈아 끼웠다. Unity에서는 **여러 Scene을 갈아 끼우는** 구조라 화면별 코드 분리가 자연스럽다.

≡ Lifecycle / Event Functions

메서드	호출 시점
Awake()	오브젝트 생성 시 한 번 (비활성 상태에서도 호출)
OnEnable()	오브젝트가 활성화될 때마다
Start()	첫 Update 직전, 한 번
Update()	매 프레임 (보통 30~120fps, 불규칙)
FixedUpdate()	고정 시간 간격(0.02초)마다 — 물리
LateUpdate()	모든 Update 후 — 카메라 추적
OnDisable()	비활성화될 때
OnDestroy()	삭제 직전

생명 주기 흐름도



🔄 흐름 요약

초기화 3단계 → 매 프레임 FixedUpdate → Update → LateUpdate 반복 → 오브젝트가 비활성화/삭제되면 OnDisable → OnDestroy로 마무리.

Awake vs Start — 둘 다 “초기화” 인데?

☰ Scene에 A, B, C 세 오브젝트가 있을 때 호출 순서

- 1 A.Awake → B.Awake → C.Awake
모든 오브젝트의 “기본 준비”가 끝남
- 2 A.Start → B.Start → C.Start
다른 오브젝트가 이미 Awake되어 있음이 보장됨
- 3 Update 루프 시작

💡 규칙

- Awake: 자기 자신의 변수 초기화, 같은 GameObject의 컴포넌트 캐싱
- Start: 다른 오브젝트와 상호작용해야 하는 초기화

Update vs FixedUpdate

두 메서드의 비교

	Update	FixedUpdate
호출 간격	프레임마다 (불규칙)	고정 (0.02초, 50회/초)
용도	입력, 일반 로직, 비물리 이동	Rigidbody2D 물리
시간 변수	<code>Time.deltaTime</code>	<code>Time.fixedDeltaTime</code>

✓ 어디에 무엇을 쓸까?

- 키보드 입력 검사 → Update
- `transform.position` 직접 변경하는 이동 → Update
- `Rigidbody2D.AddForce`, velocity 변경 → FixedUpdate
- 카메라가 캐릭터 따라가기 → LateUpdate
(캐릭터 Update가 끝난 후 따라가야 끊김 없음)

Rigidbody2D와 물리 엔진

⚙️ “이 오브젝트는 물리 법칙의 영향을 받는다”

Rigidbody2D가 붙은 오브젝트에 자동 적용: **중력**(자유 낙하), **관성**(마찰 없으면 계속 움직임), **충돌 반응**(Collider와 부딪히면 튕김).

↔ 일반 C#과 비교

일반 C#: “공이 벽에서 튕어 나오기” = `if (x < 0 || x > maxX) dx = -dx;`를 직접 작성.

Unity: Rigidbody2D + Collider2D만 붙이면 **한 줄도 안 쓰고** 튕긴다.

! 왜 FixedUpdate?

물리 엔진은 **고정 간격**으로 계산해야 정확. 프레임 속도(가변)에 맞추면 PC 성능에 따라 결과가 달라짐.

Time.deltaTime — 프레임 보정

🕒 프레임 속도와 무관한 일정 속도

```
1 void Update()
2 {
3     // BAD: speed depends on frame rate
4     transform.position += new Vector3(0.1f, 0, 0);
5
6     // GOOD: 5 units per second, regardless of fps
7     transform.position += new Vector3(5f * Time.deltaTime, 0, 0);
8 }
```

📄 Time.deltaTime이란?

“이전 프레임으로부터 지금까지 경과한 시간(초)”.

- 60fps → $\text{Time.deltaTime} \approx 0.0167\text{s}$
- 30fps → $\text{Time.deltaTime} \approx 0.0333\text{s}$

이동량 = 속도 × 시간 이므로, 속도 * Time.deltaTime을 더하면 프레임 속도와 무관하게 같은 속도로 움직인다.

객체를 만드는 두 가지 방식

일반 C# (상속/직접 작성)

```
1 class Player {  
2     public float X, Y;  
3     void Update(...) { ... }  
4     void Draw(...) { ... }  
5 }
```

Unity (컴포넌트 조합)

```
1 GameObject "Player"  
2     + Transform      (position, rotation, scale)  
3     + SpriteRenderer (image)  
4     + Rigidbody2D    (physics)  
5     + BoxCollider2D  (collision shape)  
6     + PlayerController (our script)
```

상속(Inheritance) vs 조합(Composition)

🌲 상속 = “무엇이다”(is-a)

Player는 FlyingCharacter이다 → Movable이다 → ...

새 기능 추가 = 새 자식 클래스 → **상속 지옥(deep inheritance)**

조합이 어려운 다중 능력(“건고 날고 쏘는” 캐릭터) 표현이 까다롭다.

🧩 조합 = “무엇을 가진다”(has-a)

GameObject가 Mover, Shooter, HealthComponent를 가진다.

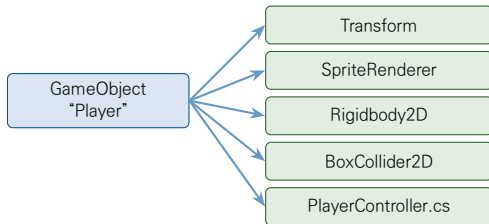
“걷는 적” 필요 → 적에 Mover만 붙이면 끝.

레고 블록처럼 기능을 조립.

★ Unity의 선택

Unity는 의도적으로 **조합**을 선택했다. 결과: 각 컴포넌트가 **한 가지 일만** 한다 → 재사용성/유연성/단순성이 함께 올라간다.

컴포넌트 모델 개념도



💡 핵심

GameObject는 빈 컨테이너일 뿐. "무엇을 할 수 있는가"는 거기에 붙어 있는 Component들의 합집합이 결정한다.

GetComponent — 다른 컴포넌트 가져오기

 PlayerController.cs

```
1 public class PlayerController : MonoBehaviour
2 {
3     Rigidbody2D rb;    // cached reference
4     SpriteRenderer sr;
5
6     void Awake()
7     {
8         rb = GetComponent<Rigidbody2D>();
9         sr = GetComponent<SpriteRenderer>();
10    }
11
12    void Update()
13    {
14        if (Input.GetKeyDown(KeyCode.Space))
15        {
16            rb.velocity = new Vector2(0, 5);
17            sr.color = Color.red;
18        }
19    }
20 }
```

Update에서 GetComponent를 호출하지 마라

⚠ Bad vs Good

```
1 // BAD: searches every frame (60+ times per second)
2 void Update() {
3     GetComponent<Rigidbody2D>().velocity = ...;
4 }
5
6 // GOOD: search once, reuse the reference
7 Rigidbody2D rb;
8 void Awake() { rb = GetComponent<Rigidbody2D>(); }
9 void Update() { rb.velocity = ...; }
```

★ 기본 성능 규칙

찾는 것은 한 번, 사용하는 것은 매번

이 패턴은 Unity 스크립트의 거의 모든 곳에서 반복된다.

Vector3 — 3D 좌표/방향

↕ Vector 연산

```
1 Vector3 p = new Vector3(2f, 5f, 0f);
2
3 // Vector arithmetic
4 Vector3 a = new Vector3(1, 0, 0);
5 Vector3 b = new Vector3(0, 1, 0);
6 Vector3 c = a + b;      // (1, 1, 0)
7 Vector3 d = a * 5f;     // (5, 0, 0)
8
9 // Useful constants
10 Vector3.zero;           // (0, 0, 0)
11 Vector3.up;             // (0, 1, 0)
12 Vector3.right;          // (1, 0, 0)
```

i 왜 묶어서 다루나?

좌표를 x, y, z로 따로 두지 않고 Vector3로 묶어 다루면

(1) transform.position = new Vector3(...) 한 줄로 위치 설정,

(2) +, -, * 같은 벡터 연산을 자연스럽게 사용 가능.

2D 게임에서는 Vector2(x, y)를 주로 사용.

transform — 모든 GameObject의 필수 컴포넌트

↔ Mover.cs

```
1 public class Mover : MonoBehaviour
2 {
3     public float speed = 3f;
4
5     void Update()
6     {
7         // 3 units per second to the right
8         transform.position += new Vector3(speed * Time.deltaTime, 0, 0);
9
10        // 90 degrees per second around Z
11        transform.Rotate(0, 0, 90f * Time.deltaTime);
12    }
13 }
```

💡 transform, gameObject는 무료

MonoBehaviour가 이미 두 속성을 가지고 있다. GetComponent<Transform>()을 부를 필요 없이 그냥 transform이라고 쓰면 된다.

Inspector에 노출되는 필드

👁 노출 규칙

```
1 public class Mover : MonoBehaviour
2 {
3     public float speed = 3f;           // shown
4     public Color color = Color.red;    // shown
5     public GameObject target;          // shown (drag-drop)
6
7     private int hiddenValue = 10;      // NOT shown
8 }
```

🏠 직렬화 규칙

- public 필드 → 자동 노출
- private 필드 → 노출 안 됨
- [SerializeField] private → **노출됨**
- public + [HideInInspector] → 숨김
- 프로퍼티(public int X { get; set; }) → **노출 안 됨**

권장 패턴: [SerializeField] private

★ 한 줄로 두 마리 토끼

```
1 public class Player : MonoBehaviour
2 {
3     // Editable in Inspector, but NOT accessible from other scripts
4     [SerializeField] private float speed = 3f;
5     [SerializeField] private int maxHp = 100;
6
7     // Read-only access from outside
8     public int CurrentHp { get; private set; }
9 }
```

✓ 두 가지를 동시에 만족

- Inspector에서 조정 가능 (디자이너 편의 / 빠른 튜닝)
- 외부에서 접근 불가 (캡슐화 유지)

OOP의 캡슐화 원칙을 Unity 환경에 맞게 가져온 표준 패턴이다.

Inspector가 있다는 것의 의미

✎ 일반 C#과의 결정적 차이

일반 C# 프로그램에서 “플레이어 속도”를 바꾸려면

→ 코드 수정 → **재컴파일** → 실행 → 확인 → 다시 수정 ...

한 번 사이클에 수십 초 걸린다.

Unity에서는 [SerializeField] float speed만 선언해 두면

→ Inspector에서 슬라이더 드래그 → **즉시 반영**

플레이 중에도 값을 바꿔 가며 “느낌”을 잡을 수 있다.

✎ 디자이너와의 협업

프로그래머가 [SerializeField]로 “조정 가능한 손잡이”를 제공하면, 디자이너/기획자가 **코드를 모르고도** 게임의 느낌을 직접 다듬을 수 있다.

(1) MonoBehaviour는 new로 만들지 마라

❌ 잘못된 vs 올바른

```
1 // WRONG: compiles, but Unity ignores it
2 PlayerController p = new PlayerController();
3
4 // CORRECT: attach to a GameObject
5 GameObject obj = new GameObject("Player");
6 PlayerController p = obj.AddComponent<PlayerController>();
7
8 // ALSO CORRECT: instantiate from a Prefab
9 GameObject p2 = Instantiate(playerPrefab);
```

⚠ new로 만들면

- Unity가 그 존재를 모름 → Update가 호출되지 않음
- transform, gameObject가 null
- Destroy(this)도 동작하지 않음

AddComponent vs Instantiate vs Prefab

🔧 세 가지의 역할

- `AddComponent<T>()` — 이미 존재하는 `GameObject`에 **컴포넌트 하나 추가**
- `Instantiate(prefab)` — Prefab을 **복제**하여 Scene에 새 `GameObject` 생성
- Prefab(**프리팹**) — 재사용 가능한 `GameObject` 템플릿(.prefab 파일)

💡 언제 무엇을 쓰나?

- 단순 일회성 → `new GameObject + AddComponent`
- 자주 만드는 복잡한 객체(총알, 적, 아이템) → **Prefab + Instantiate**

↔ 일반 C#과의 비교

일반 C#에서 “적”을 만들 때 `new Enemy(x, y)`로 코드에서 직접 생성했다면, Unity에서는 “Enemy 프리팹”을 **디자이너가 시각적으로 만들어 두고 Instantiate만 호출한다.**

(2) Update에서 무거운 작업을 하지 마라

⚠ 매 프레임 호출되므로 피해야 할 것들

- `GameObject.Find(...)` — Scene 전체 검색
- `FindObjectOfType<T>()` — 모든 오브젝트 순회
- `GetComponent<T>()` — 컴포넌트 목록 검색
- `new SomeClass()` 반복 — GC 부담
- `Debug.Log(...)` 남발 — 의외로 비싸다

★ 원칙

찾는 것은 한 번, 사용하는 것은 매번

Awake/Start에서 검색 → 필드에 캐싱 → Update에서 재사용

가비지 컬렉션(GC)이 게임에서 왜 문제인가?

문제: 프레임 끊김

GC는 메모리 청소를 위해 **프로그램을 잠시 멈춘다**.

60fps → 한 프레임 16ms뿐인데 GC가 5ms만 발생해도 **끊김(stutter)**이 보인다.

원인: 반복적인 객체 생성

`new List<>()`, `new string()`을 매 프레임 만드는 것
(`Vector3`, `Vector2`는 구조체라 GC와 무관)

해결

- 필드로 미리 만들어 두고 재사용
- `string` + 대신 `StringBuilder`
- 자주 생성/파괴되는 것 → **Object Pooling**(객체 풀) 패턴

(3) Unity의 null은 일반 C#의 null과 다르다

가짜 null (Fake Null)

```
1  GameObject obj = ...;
2  Destroy(obj); // marked for destruction
3
4  // One frame later:
5  if (obj == null) // TRUE (Unity reports null)
6      Debug.Log("destroyed");
7
8  // But the C# reference is technically not null!
9  object asObj = obj;
10 if (asObj == null) // FALSE
11     ...
```

! 실용적 규칙

항상 == null로 비교하라.

?. (null 조건) 과 ?? (null 합치기)는 "가짜 null"을 인식하지 못해 파괴된 오브젝트의 메서드를 호출할 수 있다.

(4) Coroutine — 시간이 흐르는 작업

Bomb.cs

```
1 using System.Collections;
2 using UnityEngine;
3
4 public class Bomb : MonoBehaviour
5 {
6     void Start() { StartCoroutine(ExplodeAfterDelay()); }
7
8     IEnumerator ExplodeAfterDelay()
9     {
10         Debug.Log("Bomb planted");
11         yield return new WaitForSeconds(2f);
12         Debug.Log("BOOM!");
13         Destroy(gameObject);
14     }
15 }
```

yield return 종류

- `yield return null;` — 다음 프레임까지 대기
- `yield return new WaitForSeconds(2f);` — 2초 대기
- `yield return new WaitUntil(() => isReady);` — 조건이 참이 될 때까지

Coroutine 개념: 일반 C#과의 비교

❓ 일반 C#이라면 어떻게?

“2초 후 폭발”을 일반 C#으로 짜려면

- (a) `Thread.Sleep(2000)` → 프로그램이 멈춰서 입력도 못 받음
- (b) Timer 콜백 → 콜백 지옥, 상태 관리 복잡
- (c) `async/await` → 가능하지만 게임 루프와 잘 안 맞음

✓ Coroutine의 장점

- 게임 루프에 자연스럽게 녹아들 — 각 `yield`가 “프레임 단위로 잘게 쪼개진 일”
- 상태 변수가 필요 없음 — 메서드 안의 지역 변수가 자동 보존됨
- 쓰레드 아님 — 메인 쓰레드에서 안전하게 실행, 동기화 걱정 없음

(5) 직접 그리지 않는다

Unity에서 그리기는 “컴포넌트의 일”

이미지 → `SpriteRenderer`, 3D 모델 → `MeshRenderer`, UI 텍스트 → `TextMeshProUGUI`,
선/궤적 → `LineRenderer`, 파티클 → `ParticleSystem`

스크립트의 역할

스크립트는 “어디에/어떻게” 그릴지만 결정 → `transform.position`, `sr.color`, `sr.sprite` 등을 변경. 실제 픽셀을 칠하는 것은 Unity의 렌더링 시스템.

일반 GUI와의 차이

일반 GUI: “매 프레임 Graphics 객체로 직접 그리기”를 호출. Unity: 어떤 그리기 코드도 작성하지 않는다.

Input.GetKey 세 가지 변형

세 메서드의 차이

메서드	언제 true
GetKey(K)	키 눌러 있는 동안 매 프레임 (이동)
GetKeyDown(K)	키 누른 그 프레임만 (점프, 발사)
GetKeyUp(K)	키 뗀 그 프레임만 (차징 해제)

사용 예시

```
1 void Update() {  
2     if (Input.GetKey(KeyCode.LeftArrow))  
3         transform.Translate(-5 * Time.deltaTime, 0, 0);  
4     if (Input.GetKeyDown(KeyCode.Space)) Shoot();  
5 }
```

흔한 실수

점프에 GetKey를 쓰면 Space 누르는 동안 **매 프레임 점프** → 미친 듯이 튼. “한 번 누름 = 한 번 동작”에는 GetKeyDown.

Collider와 충돌 콜백



충돌 콜백

```
1 void OnCollisionEnter2D(Collision2D c) {  
2     // physical collision (bounces)  
3     Debug.Log("Hit: " + c.gameObject.name);  
4 }  
5  
6 void OnTriggerEnter2D(Collider2D c) {  
7     // pass-through zone (no bounce)  
8     Debug.Log("Entered: " + c.gameObject.name);  
9 }
```



Collision vs Trigger

- **Collision** — 실제로 부딪히고 튕김 (벽, 바닥)
- **Trigger** — 통과는 되지만 알림은 받음 (아이템 영역)
- Collider의 **Is Trigger** 체크박스로 모드 전환

Unity가 대신 해 주는 것들

★ “코드가 짧다”고 느끼는 이유

Unity는 다음을 **자동으로** 처리한다:

- 진입점, 메시지 루프, 창 생성
- 프레임 타이머, 더블 버퍼링, 화면 갱신
- 키보드/마우스/터치 입력의 저수준 처리
- 물리 시뮬레이션 (중력, 충돌, 마찰)
- 텍스처/메시 로딩, 셰이더, 렌더링 파이프라인
- 오디오, 네트워크, 파일 시스템 추상화

✓ 우리가 작성하는 부분

오직 **내 게임의 규칙**만.

이것이 게임 엔진의 가치이고, Unity 스크립트가 짧고 “조각” 처럼 보이는 이유이다.

한눈에 보는 비교: 일반 C# vs Unity

田 같은 일을 어떻게 하나

	일반 C#/GUI 프로그램	Unity
진입점	Main()	없음 (엔진이 자동)
게임 루프	while/Timer 직접 작성	Update 자동
프레임 보정	직접 시간 측정	Time.deltaTime
화면 갱신	Invalidate/Repaint	자동 렌더링
그리기	Graphics.FillRectangle 등	SpriteRenderer
객체 모델	클래스 직접 작성	GameObject + Component
입력 처리	이벤트 핸들러	Input.GetKey 폴링
충돌 판정	Math.Abs(...) < N 직접	Collider + 콜백
값 조정	코드 수정 후 재컴파일	Inspector에서 즉시

● 동작 명세

화면에 여러 공이 떠다니다가:

- 벽에 부딪히면 **튕김** (자동, 우리 코드 없음)
- 공끼리 부딪히면 **하나로 합쳐짐** (우리 코드)

⚙ 물리 규칙 (보존 법칙)

- 면적 보존: $r_{\text{new}} = \sqrt{r_1^2 + r_2^2}$
- 질량 보존: $m_{\text{new}} = m_1 + m_2$
- 운동량 보존: $\vec{v}_{\text{new}} = \frac{m_1 \vec{v}_1 + m_2 \vec{v}_2}{m_1 + m_2}$
- 위치 = **질량 중심**

이 예제로 익히는 개념

✔ 한 번에 적용되는 개념

- MonoBehaviour, Awake/Start 초기화 패턴
- Rigidbody2D + Collider2D로 자동 물리/충돌
- OnCollisionEnter2D 충돌 콜백
- Prefab + Instantiate로 다수 객체 생성
- [SerializeField]로 Inspector 노출
- Destroy로 오브젝트 제거
- GetComponent 캐싱

Hierarchy 설계

Scene Hierarchy

```
1 Main Camera          // Unity default
2 GameManager          // empty GameObject
3   + BallSpawner.cs   // our script
4 Walls                // empty group
5   + WallTop           // BoxCollider2D
6   + WallBottom
7   + WallLeft
8   + WallRight
```

💡 설계 포인트

- 공(Ball)은 Hierarchy에 두지 않음 → **Prefab**으로 만들고 BallSpawner가 Instantiate
- 빈 GameObject Walls로 4개 벽을 묶어 정리 (SetActive, 이동을 한꺼번에)

BallPrefab 설계

공에 필요한 컴포넌트

컴포넌트	역할
Transform	위치/크기 (자동)
SpriteRenderer	원 모양 표시
CircleCollider2D	원형 충돌 영역
Rigidbody2D	물리 (관성, 충돌 응답)
Ball.cs	우리 스크립트

Rigidbody2D 설정

Gravity Scale = 0 (떨어지지 않게), Linear Drag = 0 (계속 움직이게), Collision Detection = Continuous (빠른 공 통과 방지)

Physics Material 2D Bouncy

Friction = 0, Bounciness = 1 (완전 탄성 충돌)

두 개의 스크립트

Ball.cs — 개별 공의 동작

- 시작 시 무작위 방향 발사
- 다른 공과 충돌하면 합치기

BallSpawner.cs — 전체 게임 관리

- 시작 시 N개의 공 생성

충돌 시 “누가 합치는가”?

A↔B 충돌 시 OnCollisionEnter2D가 **양쪽에서** 호출됨.

→ GetInstanceID()가 **더 작은 쪽만** 처리하도록 분기.

Ball.cs — 초기화와 충돌 감지

Ball.cs (1/2)

```
1 public class Ball : MonoBehaviour
2 {
3     [SerializeField] private float initialSpeed = 3f;
4     private Rigidbody2D rb;
5
6     void Awake() { rb = GetComponent<Rigidbody2D>(); }
7
8     void Start()
9     {
10         float angle = Random.Range(0f, 360f) * Mathf.Deg2Rad;
11         Vector2 dir = new Vector2(Mathf.Cos(angle), Mathf.Sin(angle));
12         rb.velocity = dir * initialSpeed;
13     }
14
15     void OnCollisionEnter2D(Collision2D c)
16     {
17         Ball other = c.gameObject.GetComponent<Ball>();
18         if (other == null) return; // hit a wall
19         if (GetInstanceID() > other.GetInstanceID()) return;
20         Merge(other);
21     }
22     // ... Merge() on next slide
23 }
```

Ball.cs — Merge 메서드

Ball.cs (2/2)

```
1 void Merge(Ball other)
2 {
3     float r1 = transform.localScale.x * 0.5f;
4     float r2 = other.transform.localScale.x * 0.5f;
5     float rNew = Mathf.Sqrt(r1*r1 + r2*r2); // area conserved
6
7     float m1 = rb.mass, m2 = other.rb.mass;
8     float mNew = m1 + m2; // mass conserved
9
10    Vector2 vNew = (m1*rb.velocity + m2*other.rb.velocity) / mNew;
11    Vector2 pNew = (m1*(Vector2)transform.position
12                  + m2*(Vector2)other.transform.position) / mNew;
13
14    transform.position = pNew;
15    transform.localScale = Vector3.one * (rNew * 2f);
16    rb.mass = mNew;
17    rb.velocity = vNew;
18
19    Destroy(other.gameObject); // remove the other
20 }
```

BallSpawner.cs



BallSpawner.cs

```
1 public class BallSpawner : MonoBehaviour
2 {
3     [SerializeField] private GameObject ballPrefab;
4     [SerializeField] private int count = 10;
5     [SerializeField] private Vector2 areaMin = new Vector2(-7,-4);
6     [SerializeField] private Vector2 areaMax = new Vector2( 7, 4);
7
8     void Start()
9     {
10         for (int i = 0; i < count; i++)
11         {
12             float x = Random.Range(areaMin.x, areaMax.x);
13             float y = Random.Range(areaMin.y, areaMax.y);
14             Instantiate(ballPrefab,
15                 new Vector3(x, y, 0),
16                 Quaternion.identity);
17         }
18     }
19 }
```

Editor 셋업: Step-by-Step (1/2)

⚙ Step 1-3

Step 1: 새 2D 프로젝트

Unity Hub → New Project → "2D Core"

Step 2: 벽 만들기

- Walls 빈 GameObject 생성
- 그 아래 2D Object > Sprites > Square 4개 (Top/Bottom/Left/Right)
- Scale/Position 조정 + BoxCollider2D 추가

Step 3: BallPrefab

- 2D Object > Sprites > Circle 생성 → Ball
- Rigidbody2D (Gravity 0) + CircleCollider2D 추가
- Bouncy Physics Material 2D 만들어 Collider에 할당
- Ball.cs 부착 → Project로 드래그하여 Prefab화

⚙ Step 4-5

Step 4: GameManager + Spawner

- 빈 GameObject GameManager 생성
- BallSpawner.cs 부착
- Inspector의 Ball Prefab 슬롯에 BallPrefab 드래그

Step 5: Play

- 상단 ▶ Play 버튼 클릭
- 10개의 공이 튕기다가 부딪히면 큰 공 하나로 합쳐지는 것 확인

💡 작성한 코드의 양

실제 우리가 작성한 코드는 약 60줄.
같은 게임을 일반 C#/GUI로 짤다면 수백 줄 필요.

무엇이 어디서 일어나는가

田 책임 분담표

이벤트	누가 처리하나
공 생성	BallSpawner.Start
초기 속도 설정	Ball.Start
매 프레임 위치 갱신	Rigidbody2D (자동)
벽 충돌 반사	Collider2D + Material (자동)
공-공 충돌 감지	OnCollisionEnter2D 콜백
합치기 계산	Ball.Merge
사라진 공 제거	Destroy(...)

★ 우리가 정말 작성한 부분

무작위 초기 속도 + 합치기 로직 — 그게 전부.
나머지는 모두 Unity 컴포넌트가 처리한다.

도전 과제 — 4단계 로드맵

단계별 진행

Phase	내용
1. 예제 변형	색/사운드 등 시각·청각 피드백 추가
2. 새 메커니즘	코루틴, UI, 입력으로 규칙 변경
3. 새 게임	Rigidbody/Collider/Prefab 종합 활용
4. 자유 창작	5개 이상 개념을 활용해 완성

제출

프로젝트 압축(Library/, Temp/, Logs/, obj/ 제외)

+ README.md + 화면 캡처 1~2장

Phase 1 — 예제 변형 (기초)

코드를 조금만 고쳐 결과를 크게

❶ 색상 평균 합치기

```
sr.color = (sr.color + other.sr.color) * 0.5f;
```

❷ 속도에 따른 색 변화

```
Color.Lerp(Color.blue, Color.red, |v| / maxSpeed)
```

❸ 충돌 사운드

```
AudioSource.PlayOneShot(wallHit / mergeHit)
```

벽인지 공인지 GetComponent<Ball>() == null로 분기

Phase 1의 목표

Inspector에서 즉시 반영되는 감각, “코드 한 줄로 결과가 달라짐”을 체득하는 단계.

Phase 2 — 새 메커니즘 추가 (응용)

새 컴포넌트와 코루틴 도입

❶ 마우스 클릭으로 공 추가

```
1 if (Input.GetMouseButtonDown(0)) {  
2     Vector3 mp = Input.mousePosition;  
3     mp.z = -Camera.main.transform.position.z;  
4     Vector3 w = Camera.main.ScreenToWorldPoint(mp);  
5     w.z = 0;  
6     Instantiate(ballPrefab, w, Quaternion.identity);  
7 }
```

❷ 최대 크기 분열 — 반지름 > maxRadius이면 둘로 쪼개기 (면적/운동량 보존)

❸ 코루틴 자동 스폰 — IEnumerator + yield return new WaitForSeconds(2f)

❹ HUD 점수 (싱글톤 + TextMeshProUGUI)

```
GameManager.Instance.AddScore(1)
```

Phase 3 — 새 게임 (종합 응용)

📁 과제 3.1 — 벽돌 깨기 (Block Breaker)

“공 합치기” 자산(BallPrefab, Bouncy)을 **재사용**.

- Paddle.cs — 좌우 키 이동
- Brick.cs — 충돌 시 Destroy(gameObject), 점수 +10
- Ball.cs — 화면 아래로 떨어지면 Game Over
- 모든 벽돌 사라지면 Stage Clear

🔧 과제 3.2 — 중력 토글 시뮬레이터

SPACE 키로 Physics2D.gravity = -Physics2D.gravity; 토글.

합치기 로직은 그대로 유지. 토글 시 모든 공이 한 번 반짝이도록 코루틴으로 시각화.

📄 필수: README에 비교 글

같은 기능을 일반 C#/GUI로 짤 때의 **코드 줄 수**와 본 과제에서 **우리가 작성한 줄 수**를 비교. 사라진 코드가 어떤 **컴포넌트**로 옮겨갔는지 짝지어 설명.

Phase 4 — 자유 창작

💡 5개 이상 개념을 활용한 미니 게임

필수 개념 (5개 이상)

[SerializeField], GetComponent 캐싱, Rigidbody2D, Prefab + Instantiate, OnCollisionEnter2D, Coroutine, Time.deltaTime, TextMeshProUGUI

🎮 주제 예시

- Catch the Star — 떨어지는 별 받기
- Mini Shooter — 적이 내려오고 플레이어가 발사
- Dodge — 떨어지는 공 피하기
- Top-Down Maze — 미로 탈출
- Flappy 스타일 — 클릭으로 점프, 장애물 회피

★ 팁

한 가지 메커니즘만 **확실히** 동작시킨 뒤 확장하라. Phase 1~3 자산 재사용 권장.

✔ 이번 강의에서 배운 것

개념	핵심 내용
MonoBehaviour	Unity가 인식하는 스크립트가 되기 위한 필수 상속
생명 주기	Awake/Start/Update/FixedUpdate/LateUpdate 호출 시점
컴포넌트 모델	상속이 아닌 조합 으로 GameObject 구성
GetComponent 캐싱	Update에서 매번 호출하지 말 것
Time.deltaTime	프레임 속도와 무관한 일정 속도
Inspector 직렬화	public / [SerializeField] 규칙
new 금지	AddComponent 또는 Instantiate 로
가짜 null	항상 == null 로 안전하게 검사
Coroutine	시간 흐름이 있는 작업 표현

≡ 직접 해 보기

- 1 Unity Editor에서 LifecycleDemo 스크립트를 붙이고 Console에서 Awake → Start 순서를 확인
- 2 Mover 스크립트를 작성하여 speed를 Inspector에서 조정
- 3 “화살표 키로 움직이는 캐릭터”를 작성
(Player.cs에서 키 입력 + transform.position 변경)
- 4 Bomb 코루틴을 변형하여 1초 간격으로 Debug.Log를 5번 출력 후 오브젝트가 사라지게 하라

질문?

young.min.kang@gmail.com